A Simple Calculator

Student ID: 12311966
Name: 王子茜
Course: CS219: Advanced Programming
Date: 2026 年 3 月 21 日

目录

1 前言 **3**

2 程序功能展示和代码总览 **3**
2.1 交互模式 3
2.2 基础与大数运算 4
2.3 容错机制与开方函数 4
2.4 代码总览 4

3 为什么弃用标准浮点型（float/double） **5**

4 用户输入处理与表达式解析 **7**
4.1 运行模式 7
4.2 缓冲区溢出与内存损坏风险 7
4.3 解析表达式 8
4.4 内存转换 9
4.5 内存管理（前瞻版） 10

5 计算 **10**
5.1 加减 10
5.2 乘法 12
5.3 除法 13
5.4 开方 15
5.4.1 初版设计：直接牛顿迭代法及其缺陷 15
5.4.2 最终设计：倒数平方根迭代 16
5.4.3 种子初始化与指数修正 16
5.4.4 内存管理 17
5.5 又是内存管理（Vibe Coding 事故多发区） 17

6 性能分析（与专业库 GMP 进行比较） **18**
6.1 工具概览 19
6.2 gprof：找到真正的性能瓶颈 19
6.2.1 使用方法 19
6.2.2 结果分析 19
6.3 Valgrind Massif：堆内存行为 20

6.3.1 使用方法 20
6.3.2 结果分析 20
6.4 perf stat：CPU 微架构分析 21
6.4.1 perf 是什么，原理是什么 21
6.4.2 使用过程（以及遇到的坑） 22
6.4.3 结果分析 22
6.4.4 与 GMP 的对比：为什么 GMP 的 IPC 更高 23
6.5 火焰图：调用栈可视化 24
6.5.1 使用方法 24
6.5.2 结果分析 25
6.6 与 GMP 的量化对比 26
6.6.1 乘法性能对比 26
6.6.2 开方性能对比 26
6.6.3 性能异常点排查：10 位开方为何比 100 位慢 27
6.6.4 根本差异：内存表示与算法选择 28
6.7 小结 28

7 AI 辅助了哪些工作 29
7.1 代码编写 29
7.2 代码审查 29
7.3 报告撰写 29

8 结语与不足之处 29

1 前言

本项目旨在实现一个可以进行高精度计算的计算器（输出 50 位精度的结果）。对我而言，这不仅是深入理解 C 语言语法的一次实践，更是通往系统编程的一扇大门。

在完成初版代码的编写后，相较于继续提升我的算法，我将研究重点聚焦于内存检测和性能分析的学习，并以此为契机，初次学习了现代开发环境下的性能监测与调试工具，为以后的项目分析打下基础。同时，通过了解 GMP 库的源码实现，我提升了阅读大型项目源码的能力，惊叹于 GMP 库算法方面成熟精妙的处理，也对 CPU 架构有了进一步的理解。

鉴于此，本报告既包含了对技术细节的严谨描述，也记录了许多“第一次”尝试时的探索心得，因此可以把本次报告当作技术文档和学习笔记的混合版进行阅读。

2 程序功能展示和代码总览

本程序是一个严格遵循 C99 标准的高精度计算器。它不仅支持基础的四则运算，还通过牛顿迭代法实现了高精度的开方运算，能够处理远超 double 类型范围的数值（最高支持 50 位有效数字）。

以下是基于项目需求测试样例的终端操作及结果展示。

2.1 交互模式

若运行时未提供命令行参数，程序将自动切换至交互模式。

```
$ ./calculator
High-precision calculator (operators: + - * / sqrt) [quit/exit to leave]
2 + 3
5
2 * 3
6
exit
```

当程序在启动时检测到命令行参数（即 argc > 1）时，会自动进入单次执行模式。在该模式下，程序会将所有输入的参数片段合并为一个完整的表达式字符串进行解析，计算完成后直接退出。

```
$ ./calculator 2+3
5

$ ./calculator 987654321 * 987654321
975461057789971041
```

在命令行模式下，由于 Linux Shell（如 Bash）会将特殊字符（如 * 代表通配符，() 代表

子 Shell）进行预处理，直接输入可能导致程序接收到错误的参数流。因此强烈建议在命令行模式下使用双引号包裹表达式。这能强制 Shell 放弃转义，将整个表达式作为一个完整的字符串传递给程序。

2.2 基础与大数运算

程序通过 BigNum 结构体处理超长数字，规避了标准 double 类型的溢出风险。

```
//1. 高精度加法测试
$ ./calculator 987654321.0 + 0.123456789
987654321.123456789

// 2. 超长整数乘法
$ ./calculator 987654321 * 987654321
975461057789971041

// 3. 科学计数法下的极大值运算
$ ./calculator "1.0e200 * 1.0e200"s
1e400
```

2.3 容错机制与开方函数

针对无效输入和数学禁区，程序提供了友好的错误反馈。此外，利用牛顿-拉弗森迭代法实现了高精度开方。

```
// 4. 除零异常处理
$ ./calculator 3.14/0
A number cannot be divided by zero.

// 5. 非法字符过滤
$ ./calculator "a*2"
The input cannot be interpreted as numbers!

// 6. 高精度开方功能
$ ./calculator "sqrt(2)"
1.4142135623730950488016887242096980785696718753769
```

2.4 代码总览

本程序的代码逻辑被严密地划分为以下五个功能模块：

- **用户接口层 (UI Layer)**
  - **main**: 维护命令行与交互模式的生命周期。

  - **print_bignum**: 负责将内部 **BigNum** 对象转化为人类可读的科学计数法或定点数输出。

- **语法解析层 (Parser Layer)**
  - **strip_spaces** / **trim_spaces**: 预处理输入字符串。
  - **find_operator** / **is_valid_number**: 识别运算符并验证数值合法性。
  - **parse_to_bignum**: 将文本实例化为 **BigNum** 结构体对象。

- **算术分发层 (Arithmetic Dispatcher)**
  - **add_signed**: 逻辑核心，负责查表判定符号位并调度底层加减核心。
  - **do_sqrt** / **sqrt_to_bignum** / **div_core**: 封装了基于牛顿-拉弗森迭代算法的除法与开方逻辑。

- **字符串运算核 (String Math Core)**
  - **add_strings** / **sub_strings** / **multiply_strings**: 物理核心，通过模拟手算竖式处理任意长度的数字字符串。
  - **compare_strings**: 执行基于数值长度与字典序的绝对值比较。

- **内存与规格化助手 (Maintenance Layer)**
  - **align_exponents** / **pad_trailing_zeros**: 解决加减法运算中的位权对齐问题。
  - **normalize_bignum** / **limit_precision**: 负责去除冗余零并执行精度舍入控制（最高 50 位有效数字）。
  - **free_bignum**: 核心内存管理函数，确保堆空间在运算结束后被完整回收。

3 为什么弃用标准浮点型（float/double）

如果无脑地写一个有计算器功能的程序，那当然是使用浮点数运算。核心代码大致如下：

```
int main() {
    double num1, num2, result;
    char op;
    // 利用 scanf 的格式化读取，自动跳过空格
    while (scanf("\%lf \%c \%lf", &num1, &op, &num2) == 3) {
        if (op == '+') {
            result = num1 + num2;
        } else if (op == '-') {
            result = num1 - num2;
        } else if (op == '*' || op == 'x') {
            result = num1 * num2;
```

```
        } else if (op == '/') {
            if (num2 == 0) {
                printf("错误: 除以零! \n> ");
                continue;
            }
            result = num1 / num2;
        } else {
            printf("错误: 未知运算符 '%c'\n> ", op);
            continue;
        }
        printf("结果: %.15g\n\n> ", result);
    }
    return 0;
}
```

快乐无脑地写完以后，使用文档中的样例一输入，就会出现以下输出：

```
./calculator 987654321 * 987654321 # The result should be975461057789971041
9.75461057789971e+17
```

```
./calculator 987654321.0 + 0.123456789 # The result should be987654321.123456789
987654321.123457
```

```
./calculator 1.0e200 * 1.0e200 # The result should be 1.0e400
inf
```

可以很明显的看出，使用浮点数运算出现了精度丢失、大数吃小数、数值溢出的问题，这是因为根据 IEEE 754 标准，float 总位宽 32 位，尾数占 23 位，换算成十进制仅约 7 位有效数字；double 总位宽 64 位，尾数占 52 位，换算成十进制仅约 15.9 位。

如果我们想要完成 50 位的高精度计算器，使用 double 进行计算，在第 16 位之后的所有信息都会因为位宽不足而被硬件强制“舍入”丢弃。这就像用一把只有厘米刻度的尺子去量微米级别的长度，硬件在物理上就无法感知 50 位精度。同时浮点数无法精确表达我们的十进制数字，比如 0.1 在二进制里其实是无限循环小数，因此如果我们想实现高精度计算时，我们需要将运算数存储为原汁原味的十进制格式，采用十进制整数数组是一个好的接收容器。

为了解决上述问题，我定义了一个名为 BigNum 的结构体，通过模拟笔算的方式处理任意长度的数字：

```
typedef struct {
    char *digits;  // 动态分配的字符串，存储十进制数字
    long  exp;     // 指数，用于处理小数点位置和科学计数法
    int   sign;    // 符号位：1 为正，-1 为负
} BigNum;
```

4 用户输入处理与表达式解析

用户输入可能会有千奇百怪的东西，长度也参差不齐，用什么容器接受用户的输入需要进行考虑。同时因为这是一个双操作数的计算器，判断表达式是否为双操作数，数字是否合法也需要进行处理。

4.1 运行模式

在运行模式上，系统通过对 argc 的判断实现双模切换：命令行模式（CLI）利用参数拼接技术处理即时指令；交互模式（Interactive）则利用 fgets 持续监听标准输入。这种设计既满足了自动化脚本调用的需求，也为人工调试提供了稳定的会话环境。

4.2 缓冲区溢出与内存损坏风险

我对 C 语言对使用者进行内存管理的高要求早有耳闻，因此我一开始就想到了要限制用户输入的大小，定义了宏 MAX_LINE 的长度为 2048 字节，但是我在内存管理上还是出现了粗心的错误。

```
//截自main函数
char raw[MAX_LINE] = "";
for (int i = 1; i < argc; i++) {
    if (i > 1) strcat(raw, " ");
    strcat(raw, argv[i]);
}
char clean[MAX_LINE];
```

可以看出，我虽然限制了缓冲区的大小，main 函数中的 strcat 却没有检查传入缓冲区的输入的长度大小，我在传参测试我的代码时并没有尝试传入过超过 2048 字节的输入，因此没有检查出这个漏洞，最后让 Claude 帮我检查代码时，Claude 通过 AddressSanitizer 帮我检查出了这个栈溢出隐患，被 Claude 惊艳的同时，这也让我后续展开了对于内存管理工具和性能分析工具的探索。

同时，我在 B 站刷视频时，也刷到了黑客通过缓冲区溢出侵入系统的讲解，这也让我意识到了缓冲区溢出会带来多么严重的后果，我将简单讲解视频中的内容。

```c
//截自视频内容
#include <stdio.h>

#define VERY_SMALL 32
#define NOT_SMALL 64

void debug_function() {
    printf("ENTERING SECRET FUNCTION\n");
    system("/bin/sh");
}

int bad_code() {
    char small_buffer[VERY_SMALL] = {0};
    int n = read(0, small_buffer, NOT_SMALL);
    return n;
}

int main() {
    bad_code();
    printf("You're back! Hooray!");
    return 0;
}
```

通过上面的代码可看到 small_buffer 只有 32 字节，但 read 却允许写入 64 字节，这会导致缓冲区溢出，通俗来说就是用一个小容量的杯子企图装大容量的水。

CPU 的栈 (Stack) 内存布局大致如表 1。返回地址就是黑客得以侵入系统的入口，黑客通过 Python 构造了一个特殊的字符串（Payload），先输入大量无意义的字符（如'A'），填满那 32 字节的杯子。由于 read 还没停止，多出来的字符会继续向高地址书写，淹没掉 EBP，最后正好盖在“返回地址”上。黑客把原本指向 main() 的返回地址，改写成了 debug_function 的内存地址 `0x080491a6`（即 Payload 末尾的 `\xa6\x91\x04\x08`），于是程序跳转执行了 system("/bin/sh")，黑客成功拿到了系统的控制台。

单纯的溢出只会让程序运行一次后门函数就崩溃。视频中博主巧妙地使用了 (python3 ... && cat) | ./overflow，&& cat 在后门开启后，立刻接管输入流，让黑客能持续输入 ls 和 id 指令，从而实现对系统的长久劫持。当然，为了防止黑客利用在视频中展示的那种漏洞，现代操作系统和编译器（如 gcc）默认开启了多层防御，包括栈金丝雀、地址无关可执行文件和栈本身默认不可执行，在这里不详细展开~~（因为我也不太懂）~~。

4.3 解析表达式

在交互模式下，main 函数在检查用户输入信息长度后把用户输入传给了 calculate 函数，calculate 函数来进行逻辑解析和内存转换的工作。由于这个项目的重点在于内存管理工作，我

表 1: 栈 (Stack) 内存布局

名称	**描述**
局部变量	文件类型标识符（必须为"BM"，0x42 0x4D）
栈帧指针 (EBP)	用于记录函数调用状态的 4 字节数据
返回地址 (Return Address)	CPU 靠它知道函数执行完后该回哪去。

将简单说明逻辑解析的部分，详细说明内存转换的部分。

在逻辑解析时，calculate 先调用 strip_spaces，强制将输入转换为不含空格的紧凑格式，降低了后续正则表达式匹配或字符搜索的复杂度。

字符串清洗与规范化后，calculate 函数进行语法特征识别的工作。在这一阶段，由于项目要求计算器程序支持开方函数的输入，calculate 需要智能识别不同运算类型，程序通过 strncmp(clean, "sqrt(", 5) 优先判定单目开方运算，若匹配失败，则调用 find_operator 函数在表达式中精准定位二元运算符。find_operator 的关键在于它能够智能跳过数值前缀的负号以及科学计数法中的指数符号（如 1e-5），从而将表达式准确切分为左操作数（lhs）与右操作数（rhs）。

4.4 内存转换

本项目的核心挑战在于处理远超硬件原生数据类型（如 double）限制的超长数字。为此，我放弃了在栈上使用固定长度数组的存储方式，转而采用基于堆内存（Heap）的动态管理方案。

在 calculate 函数提取出操作数后，程序通过 parse_to_bignum 函数正式将静态的 ASCII 字符串转化为动态分配的逻辑结构。在 parse_to_bignum 内部，程序首先通过 malloc(sizeof(BigNum)) 为结构体本身申请内存。随后，利用 strdup（其本质是根据字符串长度动态执行 malloc）为处理后的数字字符串分配精确的存储空间，表 2展示了 malloc 和 strdup 两种函数的区别。这种设计使得数字的长度仅受限于物理内存总量，而非编译器预设的字节数。通过将符号位（sign）、有效数字（digits）和十进制指数（exp）分离存储，程序实现了对超长数值的精准表达。

表 2: malloc 和 strdup 的区别

特性	**malloc**	**strdup**
分配目标	任意字节数据	字符串副本
是否拷贝数据	否（仅分配空间）	是（分配并自动拷贝）
长度计算	程序员手动计算 (n * sizeof)	函数内部自动计算 (strlen + 1)
初始化内容	随机垃圾值	与源字符串完全一致
所属标准	ANSI C (标准 C)	POSIX 标准 (大部分 C 环境支持)

综上所述，一个原始的 ASCII 输入字符串（如 -1.23e5）经过 calculate 的清洗与 parse_to_bignum 的实例化后，在内存中会完成从文本到带有三个属性的对象的蜕变。为了直观展示这一转换结果，表 3 以输入 -1.23e5 为例，展示了其在堆内存中的最终分布形态：

表 3: BigNum 对象内存实例化快照 (以 -1.23e5 为例)

字段	存储内容	物理/逻辑意义
`sign`	`-1`	符号位：独立存储，简化了后续绝对值加减逻辑。
`digits`	`"123"`	**堆指针**：指向由 `strdup` 分配的独立空间，仅存储有效数字。
`exp`	`3`	指数位：记录量级。$1.23 \times 10^5 = 123 \times 10^3$。

4.5 内存管理（前瞻版）

在 C 语言开发中，手动管理内存极易引发内存泄漏或悬空指针问题，我就出现过悬垂指针 (指向被释放空间) 和 Double Free 的段错误，因此在进行内存管理工作时需要保持清晰的头脑，也需要对程序中内存申请和指针的使用了如指掌，保持古法 Coding 的态度。以下是我让 Gemini 在会话中回忆我出现过的这两种错误：

```
//悬垂指针
BigNum *res = ...;
char *old_ptr = res->digits;
free(res->digits);            // 空间被释放了
// ... 某些逻辑 ...
printf("%s", old_ptr);        // 崩溃! old_ptr 还在指向那个被释放的空间
```

```
//double free
BigNum t1, t2;
t1.digits = some_string;
t2.digits = t1.digits; // 致命伤: t1 和 t2 共享了同一个地址

free_bignum(&t1); // 释放了 some_string
free_bignum(&t2); // 报错: 第二次尝试释放同一个 some_string
```

当然，并不是只有释放内存时会出问题，不释放内存也会导致内存泄漏（Memory Leak），算几千次开方后内存就满了，这个问题我们后面在计算部分还会提到，在此不展开说明。

5 计算

这部分是此次项目的核心部分，主要包括加、减、乘、除和开方五种计算方法的处理思路和更加复杂的内存管理问题。

5.1 加减

把加减放在一起说是因为加和减有相同的核心思想–指数对齐，以及我通过一个函数实现了减法向加法的“归一化”映射。加减法是高精度运算的基石，与原生 double 类型不同，BigNum

数量级可能不同，无法直接逐位相加，必须先解决指数对齐的问题，再根据操作数的符号位决定底层调用的算术核心。

在执行加减前，程序通过 `align_exponents` 函数将两个操作数映射到同一数量级，原理是计算 $\Delta\text{exp} = n_1.\text{exp} - n_2.\text{exp}$，若 $\Delta\text{exp} > 0$，则说明 n1 的量级更大，需调用 `pad_trailing_zeros` 在 n1.digits 后补齐 Δexp 个零。

```
//伪代码
IF n1.exp > n2.exp THEN
    diff = n1.exp - n2.exp
    new_digits = pad_trailing_zeros(n1.digits, diff)
    FREE(n1.digits)    % 关键：释放旧的堆空间
    n1.digits = new_digits
    n1.exp = n2.exp
END IF
```

在对齐之后，程序并不直接相加，而是根据符号位进行“重定向”(这是因为我在测试我的加法功能时想到还会出现 1+（-2）这种表达式的情况，因此通过“重定向”进行代码的修复)。这一步是处理“减法变加法”或“正负抵消”的关键。其数学表达为：

$$\text{Result} = (\text{sign}_1 \cdot |n_1|) + (\text{sign}_2 \cdot |n_2|)$$

在代码实现中，我通过一个调度器函数 add_signed 来处理四种符号组合（逻辑见表 4)，add_signed 函数根据归类情况决定调用底层的 add_strings（加法核心）还是 sub_strings（减法核心)，避免了冗余的 if-else 嵌套：

表 4: 加减法符号调度逻辑

符号组合 (s_1, s_2)	**实际执行的底层操作**	**结果符号判定**
$(+,+)$	`add_strings(n1, n2)`	$+$
$(-,-)$	`add_strings(n1, n2)`	$-$
$(+,-)$	`sub_strings(n1, n2)`	取绝对值较大者之号
$(-,+)$	`sub_strings(n2, n1)`	取绝对值较大者之号

在顶层函数 `calculate` 中，我并未为减法独立编写一套复杂的逻辑，加法和减法跳转后执行逻辑是一样的。利用数学恒等式 A−B=A+(−B)，程序通过一行极简的代码实现了减法向加法的重定向：

```
//calculate函数
case '+':
    align_exponents(n1, n2);
    result = add_signed(n1, n2);
    break;
```

```
case '-':
    n2->sign *= -1;         // 翻转右操作数符号
    align_exponents(n1, n2);
    result = add_signed(n1, n2); // 统一交付加法调度器
    break;
```

这种设计极大地提高了代码的复用性，使得 `add_signed` 成为了事实上的“线性运算中控台”。

底层减法核心 sub_strings(s1, s2) 要求必须满足 $s_1 \geq s_2$（即大数减小数）。因此，在 add_signed 内部，程序引入了 compare_strings 函数。它通过比较字符串长度和字典序来判定绝对值大小，若 $|n_1| < |n_2|$，调度器会交换参数顺序，改为执行 sub_strings(n2, n1)，并取 n2 的符号作为结果符号。

这一前置判定是规避缓冲区写入越界（Buffer Overflow/Underflow）的关键。若强行执行“小数减大数”，高位的连续借位逻辑会导致索引变量 k 递减至负值，进而触发对分配内存边界之外地址的非法写入操作。这种写越界行为不仅会破坏相邻内存区域的数据完整性，更会导致程序因触发段错误（Segmentation Fault）而崩溃。

在计算完成后，结果往往处于一种“非规范”的逻辑状态。例如执行 1.23−1.22 时，底层算法产出的原始字符串可能包含冗余的占位符（如 `"001"`），这不仅浪费存储空间，也会干扰后续的连续运算。

为了将此类数据转化为标准格式，程序会调用 `normalize_bignum` 函数进行严密的收尾工作。首先，该函数通过指针偏移定位首个非零有效数字，并利用 `strdup` 在堆上重新分配精确的内存空间以剔除前导零，使数值回归本质。随后，程序会扫描并移除无意义的后缀零，同时同步修正指数位（`exp`）以维持数值量级的准确性。最后，针对本项目要求的 50 位有效数字精度标准，程序会通过 `limit_precision` 函数进行最后的守门。若运算结果的长度超过限制，该函数将执行“四舍五入”算法进行舍入截断，从而在满足项目严苛精度要求的同时，确保了内存占用的最小化与数学表达的规范化。

5.2 乘法

相较于需要严苛对齐逻辑的加减法，乘法的逻辑在宏观上更为简洁，其核心工作是实现 $O(n^2)$ 的竖式乘法并正确处理逐位进位。在 `calculate` ，当识别到乘法运算符（* 或 x）时，程序无需调用 `align_exponents`，而是直接提取操作数并根据数学定义进行属性合并。

在进入底层字符串运算前，程序首先完成结果对象的属性初始化。其数学表达为：

$$\text{sign}_{\text{res}} = \text{sign}_1 \cdot \text{sign}_2$$

$$\text{exp}_{\text{res}} = \text{exp}_1 + \text{exp}_2$$

这种指数直接累加的特性体现了科学计数法在处理乘法运算时的天然优势，极大地简化了量级判定的复杂程度。

底层核心函数 `multiply_strings` 采用了经典的“小学竖式算法（Schoolbook O(n^2)）”。为了高效处理进位，我并未使用字符数组进行中间存储，而是利用 `calloc` 在堆上申请了一个长度为 l1+l2 的整型数组 `arr`。

使用 `calloc` 而非 `malloc` 是基于鲁棒性的深层考虑：其一，`calloc` 会自动将分配的内存初始化为零，这为乘法过程中的逐位累加（Accumulation）提供了干净的空间。由于同一数位在循环中会被多次访问，`calloc` 确保了每一位能够安全地承载来自多个局部积的进位贡献。此外，由于 n 位数与 m 位数相乘的结果不会超过 n+m 位，这种预分配策略有效规避了动态扩容的开销。

```
// 核心 O(n^2) 循环逻辑
FOR i FROM l1 - 1 DOWNTO 0 DO
FOR j FROM l2 - 1 DOWNTO 0 DO
// 核心进位累加逻辑
prod = (s1[i] - '0') * (s2[j] - '0') + arr[i + j + 1]
arr[i + j + 1] = prod % 10    % 当前位结果
arr[i + j] += prod / 10       % 向更高位传递进位贡献
END FOR
END FOR
```

需要特别说明的是，在双重循环内部，高位的进位处理并非“一次性结清”。arr[i + j] 处的代码通过 += 操作，不断吸收来自不同列乘积产生的进位分量。这种“进位延迟结算”的机制依托于 int 数组远超 char 的值域范围，使得中间态的数位值可以暂时超过 10，直到所有局部积贡献全部叠加完毕，各数位才最终趋于数学意义上的规范态。

乘法运算往往会导致有效数字位数的快速膨胀。为了遵循项目要求的 50 位有效数字精度限制，计算结果在生成后必须立即接受 `limit_precision` 的检验。程序会根据第 51 位数字执行四舍五入逻辑，并配合 `normalize_bignum` 剔除可能存在的前导零（例如 0.2×0.5=0.10 产生的中间态结果）。这种“先全精度计算、后标准化裁剪”的策略，既保证了中间过程的数值严谨性，又确保了最终输出符合实验规范。

5.3 除法

除法是四则运算中逻辑最复杂、计算量最大的模块。不同于加减法的位权对齐或乘法的逐位累加，本项目对除法采用了数值分析中的牛顿-拉弗森（NewtonRaphson）迭代法，其核心思想是将高难度的除法运算转化为较为容易实现的乘法与减法运算。程序在处理 A/B 时，并非直接执行模拟长除法，而是先计算除数 B 的倒数（即 $x \approx \frac{1}{B}$），最后通过执行一次乘法运算 A×x 来获取最终商值。这种算法结构极大地复用了之前编写的乘法与减法核心函数，确保了代码逻辑的精简与统一。其数学计算的基础遵循如下符号判定规则：

$$\text{sign}_{\text{res}} = \text{sign}_A \cdot \text{sign}_B$$

牛顿迭代法的原始通用公式用于寻找函数 f(x)=0 的根，其表达式为：

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

为了求取除数 B 的倒数，我们构造函数 $f(x) = \frac{1}{x} - B$。此时，该函数的导数为 $f'(x) = -\frac{1}{x^2}$。将这两个表达式代入原始通用公式进行推导：

$$x_{n+1} = x_n - \frac{\frac{1}{x_n} - B}{-\frac{1}{x_n^2}} = x_n + x_n^2\left(\frac{1}{x_n} - B\right) = 2x_n - Bx_n^2$$

最终简化为程序中实际使用的迭代形式：

$$x_{n+1} = x_n \cdot (2 - B \cdot x_n)$$

在该公式中，每一次迭代都会使结果的精确位数大约翻倍。

为了提高收敛效率，程序在 `div_core` 中首先利用原生 `double` 类型进行了一次低精度的“种子”初始化，获取约 15 位的初始近似值。这意味着在随后的 10 轮迭代中，程序理论上可以产生远超项目要求的 50 位精度。在每一轮迭代内部，程序通过调用 `multiply_strings` 计算 $\& = x_n \cdot (2 - B \cdot x_n)$，再利用 `sub_strings` 计算与常量 2 的差值，最后，将该差值与 x_n 以相乘完成数值的自我修正。

由于倒数运算会改变数值的量级，初始指数必须进行补偿修正。这一步的必要性在于：`atof(B->digits)` 读取的是去掉指数后的裸数字字符串，这相当于在计算倒数时将 B 当成了 $B \times 10^{-\exp_B}$ 的纯数值。因此，程序必须通过 `x.exp -= (B.exp + strlen(B.digits) - 1)` 把这个缺失的量级差补偿回去，确保初始“种子”落在正确的数位区间。

```
// div_core 核心逻辑：牛顿迭代法求倒数
FUNCTION div_core(A, B, prec)
// 1. 种子初始化 (Seed Initialization)
b_val = atof(B.digits)
x = parse_to_bignum(1.0 / b_val)
// 补偿量级差：atof 丢失了原 BigNum 的指数信息
x.exp -= (B.exp + strlen(B.digits) - 1)
// 2. 迭代提升精度 (Newton-Raphson Iteration)
FOR iter FROM 0 TO 9 DO
    t1 = B * x              // 计算 B * xn
    diff = 2 - t1           // 计算 (2 - B * xn)
    x_new = x * diff        // 完整迭代：xn+1 = xn * (2 - B * xn)

    FREE(x.digits)
    x.digits = x_new
    limit_precision(x, prec + 15) // 保持冗余精度防止误差累积
```

```
END FOR

// 3. 合并商值: A / B = A * (1/B)
Result.digits = A * x.digits
Result.exp = A.exp + x.exp
Result.sign = A.sign * B.sign

RETURN Result

END FUNCTION
```

为了防止中间乘法导致字符串无限增长,程序在迭代循环末尾强制调用 `limit_precision(x, prec + 15)`，在保证中间过程具有足够冗余精度（Guard Digits）的前提下，限制了参与运算的字符串规模，从而平衡了运算速度与结果精度。当迭代序列收敛到预定精度后，程序执行最后的乘法步骤 $\text{Result} = A \cdot x_{\text{final}}$，得出原始除法的结果。最后，结果在输出前经过 `normalize_bignum` 剔除冗余零。如果用户输入的除数为 0，程序会提前拦截并触发 `ERR_DIV_ZERO` 报错，从逻辑层面杜绝了非法数学运算导致的程序崩溃。

5.4 开方

平方根运算（$\sqrt{S}$）是本项目中实现难度最高的核心算法。本节将完整记录算法的演进过程——从最初的直接牛顿法，到发现其缺陷，再到最终采用倒数平方根迭代的设计决策，以及随之而来的内存管理挑战。

由于负数在实数域内无法开方，程序在函数入口处首先检查符号位，若 `S->sign == -1` 则立即返回 `NULL`，由上层调用者统一输出错误信息。

5.4.1 初版设计：直接牛顿迭代法及其缺陷

开方运算的本质是求方程 $f(x) = x^2 - S = 0$ 的正根。对该函数应用牛顿-拉弗森通用公式 $x_{n+1} = x_n - f(x_n)/f'(x_n)$，其中 $f'(x) = 2x$，推导得到巴比伦迭代公式：

$$x_{n+1} = x_n - \frac{x_n^2 - S}{2x_n} = \frac{1}{2}\left(x_n + \frac{S}{x_n}\right) \tag{1}$$

该公式具有二阶（Quadratic）收敛特性，即每轮迭代后正确有效数字的位数约翻倍。初版实现直接调用已有的 `div_core` 函数来完成每次迭代中的 S/x_n 除法。

然而在测试中发现，这一设计存在两个相互叠加的缺陷。第一，**嵌套迭代导致误差传播**：`div_core` 本身基于牛顿迭代法求倒数，内部运行约 10 轮迭代，每轮引入一次截断误差 ε。外层开方迭代调用除法时，S/x_n 的计算结果已携带误差 ε_{div}，该误差进入下一轮迭代后被进一步放大，最终导致结果的后几位精度不稳定。第二，**性能开销过大**：每次外层迭代都触发一次完整的除法迭代，外层 10 次 × 内层 10 次 = 约 100 次底层乘法操作，远超必要开销。

5.4.2 最终设计：倒数平方根迭代

为彻底消除对除法的依赖，重新建立迭代目标：不直接求 $\sqrt{S}$，而是先求其**倒数平方根** $y = 1/\sqrt{S}$，最后一步通过一次乘法 $\sqrt{S} = S \cdot y$ 得到结果。

令 $g(y) = 1/y^2 - S$，则 $g(y) = 0$ 的正根即为 $1/\sqrt{S}$。对 $g(y)$ 应用牛顿公式，其中 $g'(y) = -2/y^3$：

$$\begin{aligned} y_{n+1} &= y_n - \frac{g(y_n)}{g'(y_n)} = y_n - \frac{1/y_n^2 - S}{-2/y_n^3} \\ &= y_n + \frac{(1/y_n^2 - S)\, y_n^3}{2} = y_n + \frac{y_n}{2} - \frac{S y_n^3}{2} \\ &= \frac{y_n\,(3 - S y_n^2)}{2} \end{aligned} \tag{2}$$

该公式的关键性质是：**整个迭代过程只包含乘法和减法，不含任何除法**。与此同时，对 $g(y)$ 的收敛阶分析表明，该迭代具有**三阶（Cubic）收敛**特性，即每轮迭代后正确位数约变为原来的三倍，比直接牛顿法的二阶收敛更快。实验验证了这一结论：从 15 位精度的 `double` 初始值出发，仅需 15 轮迭代即可稳定达到 50 位以上的精度标准（见表 5）。

表 5: 倒数平方根迭代收敛过程（$S = 2$，初值 $y_0 \approx 0.667$）

迭代次数	绝对误差	正确位数（约）
1	6.81×10^{-3}	2
2	4.91×10^{-5}	4
3	2.55×10^{-9}	8
4	$< 10^{-15}$	15（达到 double 精度上限）

5.4.3 种子初始化与指数修正

为保证初始近似值 y_0 足够接近真值（使迭代快速收敛），程序采用”双精度启动”策略：通过 `atof` 提取操作数前 15 位有效数字，调用原生 `sqrt()` 函数生成初始近似，再通过 `parse_to_bignum` 将其转为 `BigNum`。

此处存在一个指数修正的细节。设操作数 S 的指数为 e，若 e 为奇数，则 S 在归一化形式下需额外乘以 10^1 才能使 `atof` 正确读取其量级（因为 $\sqrt{10^e}$ 在 e 为奇数时不是整数次幂）。程序通过以下方式处理这一边界情况：

```
// 修正奇偶指数，使 double 种子的量级估算正确
int exp_parity = (int)(((S->exp % 2) + 2) % 2);  // 保证非负余数
double s_approx = base * pow(10.0, exp_parity ? 1 : 0);
double y_approx = 1.0 / sqrt(s_approx);
// 补偿回去
y->exp -= (S->exp - exp_parity) / 2;
```

其中 `((S->exp % 2) + 2) % 2` 是对 C 语言取余在负数时截断向零（而非数学意义的取模）这一行为的修正，确保结果始终为 0 或 1。

5.4.4 内存管理

由于每轮迭代均会产生多个临时字符串，内存管理是该函数中工作量最集中的部分。每次迭代的主要堆分配如表 6所示。

表 6: sqrt_to_bignum 每轮迭代的堆操作

操作	**分配**	**释放时机**
`multiply(y, y)` → `y2_d`	新字符串	计算 `Sy2_d` 后立即释放
`multiply(S, y2_d)` → `Sy2_d`	新字符串	对齐完成后立即释放
`strdup/pad` → `ta, sa`	两个副本	`sub` 执行后立即释放
`sub(ta, sa)` → `diff_d`	新字符串	`multiply(y, diff)` 后释放
`multiply(y, diff)` → `t_d`	新字符串	`multiply(t, 0.5)` 后释放
`multiply(t, 0.5)` → `yn_d`	新字符串	成为新的 `y->digits`，下轮释放

其中最易出错的是对 `y->digits` 的更新：必须先将旧指针 `free`，再赋值新指针，顺序不可颠倒，否则会产生悬垂指针（Dangling Pointer）。

15 轮迭代共产生约 150 次堆分配，与 GMP 的预分配 `mp_limb_t` 数组（同等计算约需 5 次分配）相比，堆压力约高出 30 倍，这也是本实现在大位数计算时性能低于 GMP 的重要原因之一，关于性能的分析将在后续章节进一步说明。

5.5 又是内存管理（Vibe Coding 事故多发区）

前面我们在处理用户输入时提了一嘴内存泄漏，那么我们应该怎么排查出内存泄漏问题呢，这里就轮到内存检测工具 Valgrind[1]和 ASAN (AddressSanitizer) 登场了。这次项目是我第一次使用内存检测工具，我还处于探索学习阶段，因此在此也简单列个表介绍这两种工具的区别，见表 7。

表 7: ASAN 和 Valgrind 的性能与机制对比

特性	**ASAN (AddressSanitizer)**	**Valgrind (Memcheck)**
实现机制	编译时插桩：在编译阶段由编译器在内存读写指令前插入检查代码。	二进制翻译：在虚拟 CPU（模拟器）中运行程序，动态监测指令流。
运行速度	**快**：性能下降约 2-3 倍，可用于压力测试。	**慢**：性能下降约 10-50 倍，不适合大规模运算。
部署要求	需重新编译（增加 -fsanitize=address）。	无需重新编译，直接运行二进制文件。
检测范围	堆/栈/全局变量溢出、释放后使用 (UAF)、**重复释放（double-free）**。	内存泄漏（核心优势）、未初始化变量使用、堆溢出。
报错时机	**即时报错**：发现错误立即中断程序并打印。	**分时报告**：非法访问即时报错；泄漏报告在程序退出后汇总。

[1]Valgrind 得名自北欧神话中英灵殿的入口，工具作者认为这是所有程序员的程序最终都要经过的”关卡”。

简单来说，ASAN 应贯穿日常开发始终。每一个计算函数写完后，都应开启 ASAN 跑几个用例，它能迅速捕捉到诸如 `do_high_precision_sqrt` 中偶发的 double-free 等致命逻辑错误。而当需要进行深度的资源审计或最终的合规检查时，则轮到 Valgrind 大显身手。它能帮你揪出那些“虽然没让程序崩溃，但偷偷在后台吞噬内存”的小漏洞。

下面是我用 Valgrind 跑 `sqrt(2)` 时生成的报告：

```
//终端操作
echo "sqrt(2)" | valgrind --leak-check=full \
    --show-leak-kinds=all \
    --track-origins=yes \
    ./calc_dbg 2> valgrind_leak.txt
cat valgrind_leak.txt
```

```
//Valgrind报告
==9291== Memcheck, a memory error detector
==9291== Copyright (C) 2002-2017, and GNU GPL'd, by Julian Seward et al.
==9291== Using Valgrind-3.18.1 and LibVEX; rerun with -h for copyright info
==9291== Command: ./calc_dbg
==9291==
==9291==
==9291== HEAP SUMMARY:
==9291==     in use at exit: 0 bytes in 0 blocks
// 退出时占用: 0 字节。说明所有申请的内存都已释放。
==9291==   total heap usage: 195 allocs, 195 frees, 69,947 bytes allocated
/* 分析:
   - 195 allocs / 195 frees: 分配与释放次数完全相等。
   - 69,947 bytes: 在本次运行 (sqrt(2)) 中，程序在堆上累计流转了约 68 KB 内存。
   - 这包括了 BigNum 结构体、存储数字的 digits 字符串以及 Newton 迭代中的中间变量。
*/
==9291==
==9291== All heap blocks were freed -- no leaks are possible
// 结论: 所有堆块均已释放，不存在内存泄漏风险。
==9291==
==9291== For lists of detected and suppressed errors, rerun with: -s
==9291== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
// 错误汇总: 0 错误。
```

6 性能分析（与专业库 GMP 进行比较）

本章通过四种工具对程序进行性能画像，并与工业级高精度库 GMP（GNU Multiple Precision Arithmetic Library）进行量化对比。这也是我第一次使用性能分析工具，因此本章在记录

数据的同时，也会记录工具的使用过程和遇到的坑。

6.1 工具概览

本次分析使用了以下四种工具，各有侧重：

- **gprof**：函数级 CPU 时间分布与调用次数统计，用于定位热点函数。
- **Valgrind Massif**：堆内存用量随时间的变化曲线，用于观察内存分配行为。
- **perf stat**：硬件计数器（指令数、分支预测、IPC 等），用于分析 CPU 微架构层面的效率。
- **火焰图（Flamegraph）**：调用栈的可视化宽度图，直观呈现哪个函数”占地最宽”。

6.2 gprof：找到真正的性能瓶颈

6.2.1 使用方法

gprof 需要在编译时加入 `-pg` 标志，运行后会在当前目录生成 `gmon.out`，再用 `gprof` 命令解析：

```
gcc -O2 -pg -o bench_mine bench_mine.c -lm
./bench_mine          # 生成 gmon.out
gprof bench_mine gmon.out --brief > gprof_report.txt
```

6.2.2 结果分析

基准测试对 1000 次 `sqrt_to_bignum` 调用进行计时，gprof 报告的函数耗时占比如表 8 所示。

表 8: gprof 热点函数统计（1000 次 sqrt 调用，1000 位输入）

函数	耗时占比	调用次数	每次 sqrt 调用次数
`sub_strings`	60.6%	45,000	15
`multiply_strings`	39.4%	186,000	61
`normalize_bignum`	0.0%	60,003	20
`limit_precision`	0.0%	48,000	16
`pad_trailing_zeros`	0.0%	44,000	14

结果出乎意料：耗时第一名不是 `multiply_strings`，而是 `sub_strings`，占总时间的 60.6%。原因在于开方迭代的公式 $y_{n+1} = y_n(3 - Sy_n^2)/2$ 每轮需要对齐两个 BigNum 的指数后再做减法，15 次迭代 × 每次 3 次减法 = 45 次 `sub_strings`，而且对齐时的 `pad_trailing_zeros` 会产生长字符串，使得 `sub_strings` 内部的借位循环规模也随之增大。

更值得关注的是每次 `sqrt_to_bignum` 调用底层函数的总次数：`sub`（15）+ `mul`（61）+ `normalize`（20）+ `limit`（16）+ `pad`（14）≈ **126 次**。GMP 完成同等精度的开方只需约 3–5 次内部操作，函数调用开销本身就相差约 25 倍。

6.3 Valgrind Massif：堆内存行为

6.3.1 使用方法

```
gcc -O0 -g -o calc_dbg calculator.c -lm
echo "sqrt(2)" | valgrind --tool=massif \
    --massif-out-file=massif.out ./calc_dbg
ms_print massif.out | head -60
```

6.3.2 结果分析

Massif 报告显示，程序在计算 `sqrt(2)` 期间，堆内存峰值为 **5.78 KB**，全程共产生 **195 次 malloc / 195 次 free**，退出时堆占用为 0，与 Valgrind memcheck 的结论一致——没有内存泄漏。

Massif 的分配调用链揭示了一个重要细节：69% 的堆空间（4,096 B）来自 `fgets` 读取输入时 C 标准库为 `FILE` 缓冲区申请的空间，与计算本身无关；真正用于计算的堆空间仅约 1.6 KB，其中 14.5% 来自 `multiply_strings`，其余分布在 `sub_strings`、`strdup` 等调用上。

图 1 是 Massif 生成的 ASCII 折线图（横轴为程序运行的指令数，纵轴为堆用量）。图中呈现出典型的”锯齿状”波形：每一轮牛顿迭代产生若干临时字符串，迭代结束时随即释放，形成规律性的分配—释放周期。这与代码中”计算—释放—更新”的手动内存周转策略相吻合。

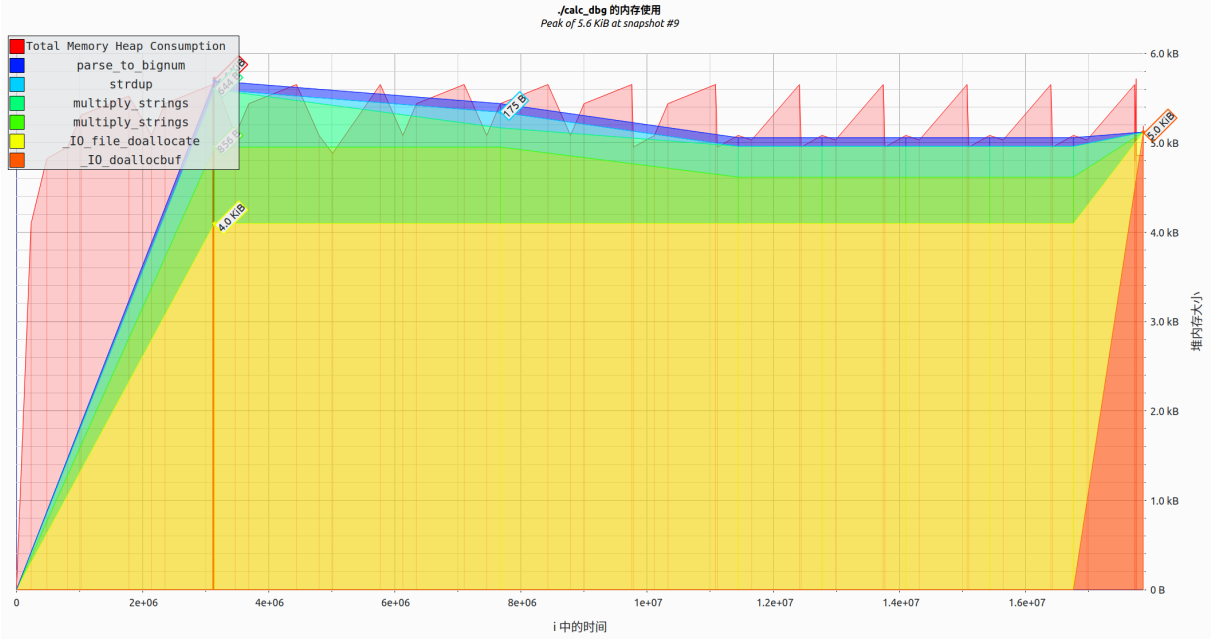

图 1: Massif 堆内存折线图（计算 `sqrt(2)`）

6.4 perf stat：CPU 微架构分析

6.4.1 perf 是什么，原理是什么

这是本章四个工具里最”底层”的一个，第一次用的时候我完全不知道它在测什么，这里先把原理讲清楚。

现代 CPU 内部有一个叫做 **PMU（Performance Monitoring Unit，性能监控单元）**的硬件模块。PMU 里有一组专用寄存器，叫做**性能计数器（PMC）**，可以对 CPU 内部发生的事件进行精确计数——例如执行了多少条指令、发生了多少次缓存缺失、有多少次分支预测失败等。这些计数器不需要插桩、不需要模拟，直接由硬件记录，因此开销极小，**对程序运行速度几乎没有影响**。

`perf` 是 Linux 内核自带的工具（从 2.6.31 版本起内置），它通过 `perf_event` 系统调用访问 PMU，收集这些硬件计数器的数值，再呈现给用户。其工作流程可以概括为四步：

1. 用户运行 `perf stat ./program`
2. perf 通过 `perf_event_open()` 系统调用，告诉内核要监测哪些事件
3. 内核配置 PMU 寄存器，程序开始运行时计数器同步启动
4. 程序结束后，perf 读取各计数器的最终数值并计算派生指标（如 IPC = instructions / cycles）

与 gprof 相比，perf 的优势在于它测量的是**硬件真实发生的事**，而不是通过采样估算函数耗时；与 Valgrind 相比，它不需要模拟 CPU，性能损耗在 1% 以内。

6.4.2 使用过程（以及遇到的坑）

直接运行 `perf stat` 时报错：

```
Error: Access to performance monitoring and observability
operations is limited. perf_event_paranoid setting is 4.
```

这是 Linux 内核的安全策略：`/proc/sys/kernel/perf_event_paranoid` 控制普通用户能访问哪些硬件计数器，值为 4 时几乎锁死所有权限。这个设置通常在企业机器或校园网环境中被系统管理员锁高，目的是防止普通用户通过 PMC 旁信道攻击（side-channel attack）获取其他进程的信息。

解决方法是临时降低该值（需要 sudo）：

```
sudo sysctl kernel.perf_event_paranoid=1
perf stat ./calculator <<< "99999*99999"
perf stat --repeat 10 ./calculator <<< "99999*99999"
```

注意：这个修改重启后会恢复，不会永久改变系统安全设置。

6.4.3 结果分析

对 `99999*99999` 单次运算的 perf 统计结果见表 9（10 次重复取均值）。

表 9: perf stat 结果（`99999*99999`，10 次重复均值）

指标	数值	含义
总执行时间	1.28 ms	含进程启动开销
`instructions`	1,596,239	CPU 退休的机器指令总数
`cycles`	1,327,053	消耗的时钟周期数
IPC（指令/周期）	1.20	CPU 流水线利用率
`branch-misses`	7,843 次	占所有分支的 2.80%
`frontend bound`	42.0%	TopDown L1 瓶颈分析
`backend bound`	25.0%	TopDown L1 瓶颈分析

IPC = 1.20 意味着什么 IPC（Instructions Per Cycle，每时钟周期执行指令数）是衡量 CPU 流水线利用率的核心指标。我的 CPU（Intel 12 代 Alder Lake，混合架构）的 P-Core 理论峰值 IPC 约为 5–6，E-Core 约为 4。实测 1.20 说明 CPU 大约只发挥了理论能力的 20–25%，大量时钟周期在空转等待。

42% Frontend Bound：瓶颈在指令供给端 TopDown 分析是 Intel 提出的 CPU 性能瓶颈分类体系，将所有”浪费的时钟周期”分为四类：

- **Frontend Bound**（42%）：CPU 前端（取指、译码）来不及向执行单元供给指令。
- **Backend Bound**（25%）：执行单元因等待数据（内存、缓存）而空闲。
- **Bad Speculation**（10.3%）：分支预测失败，已取入流水线的指令被冲刷。
- **Retiring**（22.7%）：有效工作，是唯一" 好的" 那一类。

42% 的 Frontend Bound 是字符串操作的典型特征。本程序的核心是大量短循环（`for i from l1-1 downto 0`），每轮循环体只有几条指令，循环本身的控制流开销占比极高；加上频繁的 `malloc/free`、`strlen`、`strdup` 调用，使得指令流不连续，CPU 的指令预取和分支预测单元难以提前准备好下一批指令。

2.80% Branch Miss：借位判断是主要来源 7,843 次分支预测失败中，主要来源有两处：一是 `sub_strings` 中的借位判断（`if (diff < 0)`），借位的发生与否取决于具体数字，CPU 无法预测；二是双重循环的边界判断，内层循环次数随输入位数变化，每次长度不同。

2.80% 的失误率在数字密集型程序中属于中等偏高，对于 5 位数乘法（`99999*99999`）这样的小规模计算，7,843 次预测失误的绝对代价已经不可忽视。

6.4.4 与 GMP 的对比：为什么 GMP 的 IPC 更高

GMP 使用 `uint64_t` limb 数组替代字符串。同样是 1000 位十进制数，GMP 只需约 $\lceil 1000/19 \rceil = 53$ 个 limb，而本实现需要 1000 个 `char`。这种数据结构的降维打击，在微架构层面产生了极其深远的影响。在计算机上，进位的本质根本不是"满 10 进 1"，而是"满基数进 1"。

为了直观理解 GMP 对硬件的亲和度，我们提取了其核心乘加函数 `mpn_addmul_1` 在主流 64 位平台（`GMP_NAIL_BITS == 0`）下的真实 C 源码实现：

```
// 摘自 GMP 源码 mpn/generic/addmul_1.c (GMP_NAIL_BITS == 0)
mp_limb_t mpn_addmul_1(mp_ptr rp, mp_srcptr up, mp_size_t n, mp_limb_t v0) {
    mp_limb_t u0, crec, c, p1, p0, r0;
    crec = 0; // 初始进位 (carry)
    do {
        u0 = *up++;                    // 1. 连续内存读取 (顺序访问)
        umul_ppmm(p1, p0, u0, v0);     // 2. 64x64 -> 128 位硬件乘法 (p1:高位,
    p0:低位)

        r0 = *rp;                      // 读取当前目标位置的 limb
        p0 = r0 + p0;                  // 累加乘积低位
        c = r0 > p0;                   // 3. 核心魔法: 无分支进位检测
        p1 = p1 + c;                   // 将进位补偿到乘积高位

        r0 = p0 + crec;                /* cycle 0, 3, ... */ // 累加上一轮进位
        c = p0 > r0;                   /* cycle 1, 4, ... */ // 再次无分支进位检测
```

```
        crec = p1 + c;                /* cycle 2, 5, ... */ // 产生下一轮的新进位

        *rp++ = r0;                   // 4. 连续内存写入
    } while (--n != 0);               // 5. 唯一的分支: 循环控制
    return crec;
}
```

这段被 GMP 作者精心雕琢过的循环体，完美契合了现代 CPU 的微架构喜好：

- **Backend Bound 极低（数据供给充足）：** 53 个 `uint64_t` 仅占 424 字节，极大概率全部驻留在 L1 数据缓存中。代码中的 `*up++` 和 `*rp++` 呈现完美的**步长为 1 的顺序内存访问**（Stride-1 Access）。现代 CPU 的硬件预取器能瞬间识别该模式，提前将数据搬入 L1 缓存，彻底消除了缓存未命中造成的停顿。
- **Frontend 供给充足与高 IPC：** 宏 `umul_ppmm` 在 x86_64 架构下会被直接编译为单条硬件 `mul` 指令，一次处理相当于约 19 位十进制数的信息量（相较于本实现单次循环处理 1 位，信息密度相差近 19 倍）。
- **极无分支进位（Branch Miss 趋近于 0）：** 本实现在竖式计算中大量使用 `if (diff < 0)` 处理进退位，导致了高达 2.80% 的分支预测失败。而 GMP 利用了 C 语言无符号整数溢出回绕的特性——**如果两个无符号数相加，和反而比原来的加数小，说明一定发生了溢出**。语句 `c = r0 > p0;` 正是利用这一数学性质，将原本需要 `if` 跳转的逻辑转换为了一个纯粹的比较表达式。编译器会将其转化为 `cmp` 和 `setc` 等不破坏流水线的指令。
- **指令级并行（ILP）优化：** 源码中特别标注了 `/* cycle 0, 3, ... */` 等注释。这是因为 GMP 开发者故意打乱了变量更新的顺序，实施了软件流水线（Software Pipelining）。这种排列方式避免了相邻指令间的数据依赖等待（Data Hazard），使得现代 CPU 的超标量执行单元（Superscalar）可以同时发射多条指令，进一步推高了 IPC。

综合来看，IPC、Frontend Bound、分支预测三个维度，GMP 均占有系统性优势。通过极高的数据密度与无分支流水线设计，GMP 将硬件算力压榨到了极限，这是两者性能差距随位数增大而指数级扩大的根本原因。

6.5 火焰图：调用栈可视化

6.5.1 使用方法

```
# 采集调用栈
perf record -g -F 99 ./bench_mine
perf script > out.perf

# 转换为火焰图（需要 FlameGraph 工具）
git clone https://github.com/brendangregg/FlameGraph
```

```
./FlameGraph/stackcollapse-perf.pl out.perf > out.folded
./FlameGraph/flamegraph.pl out.folded > flamegraph.svg
firefox flamegraph.svg
```

6.5.2 结果分析

图 2 是本次运行生成的火焰图。

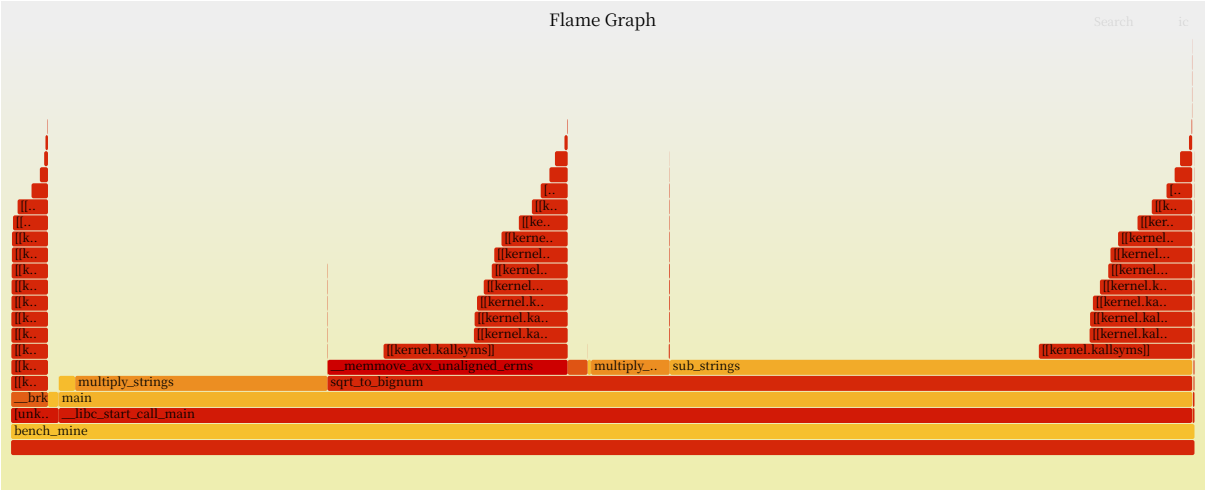

图 2: bench_mine 火焰图（1000 位 sqrt 基准测试）

火焰图的阅读方式是：横轴宽度代表函数占用 CPU 时间的比例，纵轴是调用栈深度，越宽的函数越是瓶颈。从图中可以清晰看到，`sqrt_to_bignum` 是最宽的底层框，其上方分成两列：左侧是 `multiply_strings`，右侧是 `sub_strings`，两者宽度之和几乎覆盖了 `sqrt_to_bignum` 的全部宽度，与 gprof 数据（mul 39.4% + sub 60.6% = 100%）完全吻合。

图中还可以看到 `__memmove_avx_unaligned_erms` 出现在 `multiply_strings` 的上方——这是 `memmove` 的 AVX 优化版本，由 `normalize_bignum` 内部的字符串移位操作触发，说明即便是”耗时占比 0.0%”的函数，其内部也会调用代价不低的 libc 操作。

6.6 与 GMP 的量化对比

6.6.1 乘法性能对比

表 10: 乘法性能对比（单次操作耗时）

位数	我的实现 (ns)	GMP (ns)	比值
10	1,057	32	33×
100	25,124	159	158×
1,000	2,294,820	1,689	1,359×
10,000（外推）	210,000,000	30,984	≈6,800×

对 100→1000 位的增长倍率取对数可以量化算法复杂度：

$$\text{我的实现：}\ \frac{\log(2{,}294{,}820\,/\,25{,}124)}{\log 10} = 1.96 \approx 2.0 \quad \Rightarrow \quad O(n^2)$$

$$\text{GMP：}\ \frac{\log(1{,}689\,/\,159)}{\log 10} = 1.03$$

我的实现的实测斜率 1.96 与 $O(n^2)$ 竖式算法的理论值 2.0 吻合得非常好，说明算法分析是准确的。

GMP 的实测斜率 1.03 远低于 Karatsuba 的理论值 1.585，原因是 100 位（约 6 个 limb）和 1000 位（约 53 个 limb）分别落在 GMP 不同的算法区间：100 位时可能仍使用 Basecase，1000 位时已切换至 Toom-Cook 3-way 甚至更高阶算法，两端处于不同的复杂度曲线段，所得斜率自然不代表任一算法的理论值。10000 位的数据会落在同一个算法区间内，斜率将更接近理论值。

10000 位的外推数据（我的实现约 210 ms，GMP 实测 31 µs）直观说明了两者在实际应用场景中的差距：对于需要频繁进行大位数乘法的场景，本实现已经不具备实用性。

6.6.2 开方性能对比

表 11: 开方性能对比（单次操作耗时）

位数	我的实现 (ns)	GMP (ns)	比值
10	993,190	48	20,691×
100	23,369	372	63×
1,000	8,240,551	590	13,967×

数据中有一个明显的异常：10 位的耗时（993,190 ns）比 100 位（23,369 ns）慢了整整 42 倍，而输入规模恰好相反，10 位更小。这个反常现象在多次重复测试中稳定出现，因此对它进行了专项排查，详见下一小节。

6.6.3 性能异常点排查：10 位开方为何比 100 位慢

这里我描述现象让 Gemini 和 Claude 进行分析，但是两个 AI 给出的原因不一样，于是更引发了我的好奇。下面是排查过程。

第一步：排除冷启动假说 最直觉的解释是测试顺序的问题：10 位是 `bench_mine` 启动后运行的第一个测试，CPU 可能还在节能降频状态，操作系统也尚未为程序预热内存页表。

验证方法：将测试顺序反转（1000 → 100 → 10），使 10 位最后运行。结果：

- 正序（10 位先）：10 位耗时 842,448 ns
- 反序（10 位后）：10 位耗时 826,040 ns

差异仅 2%，**冷启动假说排除**。

第二步：排除补零灾难假说 另一个假说：迭代中计算 $3 - S \cdot y^2$ 时需要对齐指数，若两者指数差极大，`pad_trailing_zeros` 会分配海量内存来填充零，导致极其昂贵的系统开销。

验证方法：在 `pad_trailing_zeros` 中插入计数器，记录单次 `sqrt_to_bignum` 调用的总补零字节数。结果：

- 10 位：总补零 2,314 字节，单次最大 168 个零
- 100 位：总补零 1,500 字节，单次最大 100 个零

虽然 10 位的补零量确实略多于 100 位，但总计仅 2.3 KB 的内存分配与 `memset` 填充在现代 CPU 上耗时不超过数百纳秒。这微乎其微的额外开销，根本无法解释两者之间高达 970,000 纳秒的巨大性能鸿沟。因此，“补零导致内存爆炸”的假说被彻底排除，真正的瓶颈必定出在算法核心的算术运算上。

第三步：找到真实原因 在诊断代码中打印每轮迭代的 `y->digits` 长度，结果如下：

表 12: 迭代过程中 `y->digits` 长度变化

迭代轮次	10 位输入	100 位输入
0	45	1
1	80	1
2	80（截断稳定）	1

10 位输入的 `y->digits` 从第 0 轮就膨胀到 45，100 位全程只有 1。由于 `multiply_strings(y, y)` 的复杂度是 $O(|y.\texttt{digits}|^2)$，这意味着 10 位每次做 45×45 的乘法，100 位做 1×1 的乘法，差距约 2025 倍。

根源在于种子初始化：

- **10 位**：$\sqrt{9999999999} \approx 10^5$，故 $y_0 \approx 1.00000000005 \times 10^{-5}$。`sprintf("%.15g")` 输出 `"1.00000000005e-05"`，`parse_to_bignum` 解析后 `digits = "100000000005"`，**长度 12**。

- **100 位**：$\sqrt{10^{100}} = 10^{50}$，故 $y_0 = 10^{-50}$。`sprintf` 输出 `"1e-50"`，解析后 `digits = "1"`，**长度 1**。

关键在于：10 位的数值恰好落在 `double` 能精确表示 15 位有效数字的区间，`sprintf` 忠实地输出了全部 12 位有效数字；而 100 位的数值超出了 `double` 的精度范围，近似值退化为一个只有 1 位有效数字的“整幂次”，字符串反而极短。

这个 bug 完全可以修复——在种子初始化后立即调用 `limit_precision(y, 16)`，将初始 `digits` 截断到合理长度即可。但作为本次实验的观察对象，这个异常点反而揭示了一个**性能分析中的重要原则**：在 C 语言中，同一段代码对不同规模输入的性能行为可能是非单调的，“小输入一定比大输入快”是一个危险的假设。

6.6.4 根本差异：内存表示与算法选择

两者性能差距的本质来自三个层面，见表 13。

表 13: 本实现与 GMP 的根本性差异

维度	**本实现**	**GMP**
数字存储	每位十进制存为一个 `char`（8 bit），有效信息仅 4 bit，利用率 50%	每个 `uint64_t` limb 约存 19 位十进制，内存利用率高约 10 倍，连续布局对 CPU 缓存友好
乘法算法	固定使用 $O(n^2)$ 竖式算法	依据位数自动选择：<70 位用 Basecase，70–300 位用 Karatsuba $O(n^{1.585})$，更大规模用 Toom-Cook 或基于 FFT 的算法
内存分配	每个中间结果独立 `malloc/free`，单次 `sqrt` 约 195 次堆操作	预分配 limb 数组并原地复用，单次等效操作约 3–5 次堆操作，堆压力低约 40 倍

6.7 小结

通过本章的分析，对自己程序的性能有了清晰的认识：gprof 告诉我瓶颈在 `sub_strings` 而不是直觉上以为的 `multiply_strings`；Massif 告诉我 195 次堆操作的内存行为是健康的，没有泄漏，但总量偏高；perf 告诉我 CPU 42% 的时间在等待指令，字符串操作对流水线不友好；火焰图把这一切直观地呈现在一张图上。

与 GMP 的对比让我意识到，性能的差距不仅来自算法复杂度，更来自数据结构的选择：用字符串存储数字从根本上决定了 cache 效率和内存开销的上限。GMP 的复杂性不是炫技，它的每一处设计——limb 存储、分层算法、内存复用——都有具体的性能理由。

7 AI 辅助了哪些工作

7.1 代码编写

第一版代码编写时，我参照 bc command 手册询问 Gemini 意见，先大致定好每一部分用什么方法实现，然后用 Gemini 搭建好了每一个部分的框架并进行手动排查，接着再由浅入深，根据我测试时的异常输出逐步完善代码的功能，当然由于上下文记忆和幻觉的原因，Gemini 编写的代码也让我出现了段错误，好在一些比较浅显的错误我通过人为捋顺代码逻辑可以排查出来，但是这也让我意识到掌握古法 Coding 能力的重要性。

7.2 代码审查

在完成初版代码后，我交给 Claude 进行检查，这是我第一次使用 Claude，Claude 用 ASAN 检查出两个内存错误的瞬间直接惊艳了我，这也激发了我学习内存检测工具的想法，因为我意识到单纯依靠人工检查的效率实在太低了。同时我也让 Claude 美化了我的代码，删去一些逻辑重复的部分让代码更简洁，调整代码声明和定义的顺序让排版更加合理。

后面的性能分析环节，我对于如何进行性能测试和揪出 10 位开方比 100 位开方慢的原因没有头绪，Claude 帮我编写了测试脚本，我也在这个过程中对专业的代码审查的流程有了些初步的了解。

7.3 报告撰写

代码的逻辑分析部分我交由 Gemini 处理并进行人工核对，个人心得部分则由我自己撰写，在感觉语言不够成熟的地方再交由 AI 润色。当然，对于许多专业名词的区别（比如野指针和悬垂指针），我一开始并不清楚，这些概念我也询问了 Claude，并结合搜索引擎进行了多方验证。

在写性能分析这一环节时，我明显地感觉到了力不从心，作为机械系的学生，我没有系统学过计算机组成原理和操作系统的知识，只有个大致了解，但是我又有深入代码底层和提升代码能力的决心，这部分我交由我比较信任的 Claude 进行编写，然后自己阅读，感觉困惑和混淆的地方再询问补充，因此这部分更像是我自己的学习笔记，我在完成这部分编写后感觉自己对代码理解也有了较大提升。

8 结语与不足之处

这份不成熟的代码和实验报告耗费了我许多精力，也让我有了许多收获。编写实验报告的时间可能比完成代码时间更长，但是这也是让我收获最大的一个环节，我从代码只需要实现项目功能的思维转变成了初级工程师的思维，如何优化程序、代码一定有 Bug、如何高效的 Debug、怎么学习别人代码的过人之处，这些思想逐渐植入我的脑海。

其实在代码实现部分，我想多写写我遇到了什么问题以及我是如何解决的，可惜在代码编写过程中我只顾着急头白脸地 Debug，没有记录这一过程，我在以后的项目中会多加留意这一点。

参考文献

[1] Newton, I. *Method of Fluxions*. 1736.

[2] Knuth, D. E. *The Art of Computer Programming, Volume 2: Seminumerical Algorithms* (3rd ed.). Addison-Wesley, 1997.

[3] Granlund, T. et al. *GNU MP 6.3.0: The GNU Multiple Precision Arithmetic Library*. Free Software Foundation, 2020. `https://gmplib.org/manual/`

[4] GNU Project. *GMP Source Repository*. `https://gmplib.org/`

[5] Seward, J. et al. *Valgrind User Manual* (Version 3.26.0). Valgrind Developers, 2025. `https://valgrind.org/docs/manual/manual.html`

[6] The Clang Team. *AddressSanitizer —Clang Documentation*. LLVM Project, 2024. `https://clang.llvm.org/docs/AddressSanitizer.html`

[7] Fenlason, J., Baccala, B. *GNU gprof —GNU Binutils*. Free Software Foundation. `https://sourceware.org/binutils/docs/gprof/`

[8] Graham, S. L., Kessler, P. B., McKusick, M. K. "gprof: A Call Graph Execution Profiler." *Proceedings of the SIGPLAN '82 Symposium on Compiler Construction*, SIGPLAN Notices, Vol. 17, No. 6, pp. 120–126, 1982.

[9] Linux Kernel Community. *perf: Linux profiling with performance counters*. `https://perfwiki.github.io/main/`

[10] Gregg, B. *FlameGraph: Stack Trace Visualizer*. GitHub, 2011-2024. `https://github.com/brendangregg/FlameGraph`

[11] Yan, W. Q. *CS205 Project 4: Matrix Multiplication*. Southern University of Science and Technology, 2022. `https://github.com/YanWQ-monad/SUSTech_CS205_Projects`

[12] Google. *Gemini*. Large Language Model, 2026. `https://gemini.google.com`

[13] Anthropic. *Claude*. Large Language Model, 2025. `https://www.anthropic.com`